

CHAPTER 6

ABSTRACT

This chapter covers advanced Express.js concepts like file upload using Multer, dynamic templates with EJS, and email functionality using Nodemailer.

Multer

Multer is a node.js middleware for handling multipart/form-data, which is primarily used for uploading files. Multer adds a file or files object to the request object. The file or files object contains the data of files uploaded via the form.

Installation

```
npm install multer
```

File information

Each file contains the following information:

Key	Description
fieldname	Field name specified in the form
originalname	Name of the file on the user's computer
encoding	Encoding type of the file
mimetype	Mime type of the file
size	Size of the file in bytes
destination	The folder to which the file has been saved
filename	The name of the file within the destination
path	The full path to the uploaded file

Storage(DiskStorage)

- ✓ The disk storage engine gives you full control on storing files to disk.
- ✓ There are two options available, destination and filename. They are both functions that determine where the file should be stored.
 - **destination** is used to determine within which folder the uploaded files should be stored. This can also be given as a string (e.g. '/tmp/uploads'). If no destination is given, the operating system's default directory for temporary files is used.
 - **filename** is used to determine what the file should be named inside the folder. If no filename is given, each file will be given a random name that doesn't include any file extension.

```
var storage = multer.diskStorage({
```

```

destination:"single",
filename: function (req, file, cb) {
  cb(null, file.originalname)
}
})

```

multer(opts)

Multer accepts an options object, the most basic of which is the **dest/storage** property, which tells Multer where to upload the files. In case you omit the options object, **the files will be kept in memory and never written to disk.**

The following are the options that can be passed to Multer.

Key	Description
dest / storage	Where to store the files
fileFilter	Function to control which files are accepted
limits	Limits of the uploaded data
preservePath	Keep the full path of files instead of just the base name

- ✓ If you want more control over your uploads, you'll want to use the storage option instead of dest.
 - **.single(fieldname)**
 - Accept a single file with the name fieldname. The single file will be stored in **req.file**.
 - **.array(fieldname[, maxCount])**
 - Accept an array of files, all with the name fieldname. Optionally error out if more than maxCount files are uploaded. The array of files will be stored in **req.files**.

Create an HTML Form

Create an HTML form that allows users to select and upload a file to your server. The form will be submitted using the POST method and will have an encoding type of "multipart/form-data" to handle file upload:

To create the HTML form, you will use the **< form >** element with the following attributes:

- ✓ **action:** This attribute specifies the URL or route where the form data will be sent when the form is submitted

- ✓ **method**: This attribute specifies the HTTP method to be used when submitting the form. For file uploads, you should use the POST method, as it allows larger amounts of data to be sent. So, set the method attribute to "POST".
- ✓ **enctype**: This attribute specifies the content type used to submit the form data. For file uploads, you need to set the enctype attribute to "multipart/form-data". This encoding type is necessary for browsers to properly handle file uploads.

Inside the form, you will add an **<input>** element of type "file". This element allows users to select a file from their local machine. Give the input element a **name** attribute so that it can be identified when the form is submitted.

Finally, you can add any additional form fields or submit buttons as needed. When the form is submitted, the selected file(s) will be sent to the specified route for handling.

Set Up Multer Middleware

Multer is a middleware for handling multipart/form-data requests, specifically designed for file uploads in Node.js.

- ✓ First, import the **multer** module using **require('multer')**. This ensures that you have access to the Multer functionality.
- ✓ Next, define the storage configuration for uploaded files using **multer.diskStorage()**. This configuration determines where the uploaded files will be stored on the server. It takes an object with two functions: **destination** and **filename**.
- ✓ The destination function specifies the directory where the uploaded files will be saved.
- ✓ The filename function determines the name of the uploaded file.
- ✓ We append the original name of the file using **file.originalname** to maintain some context about the uploaded file. You can modify this function to generate filenames based on your specific needs.
- ✓ After setting up the storage configuration, you create an instance of Multer by calling **multer({ storage })**, passing in the **storage** configuration object. This creates the Multer middleware that you can use in your Express application to handle file uploads.

Example1

Write an express js script that accepts **single file** to be uploaded using the multer middleware and saves the file to the specific directory called **"lju"**.

m1.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
//CREATE EXPRESS APP
```

```
const app = express();
app.use(express.static(__dirname,{index: 'm1.html'}));
```

// SET STORAGE

```
var store = multer.diskStorage({
  destination: "lju",
  filename: function (req, file, cb) {
    cb(null, file.originalname)
  }
})
var upload = multer({ storage: store })
app.post('/uploadfile', upload.single('mypic'), (req, res) => {
  const file = req.file
  if (file) {
    res.send("<h1>File <span style='color:red'>"+ file.originalname + "</span> has been
uploaded in <span style='color:red'>" + file.destination + " </span>folder")
  }
})
app.listen(6788);
```

m1.html

```
<html>
  <form action="/uploadfile" method="post" enctype="multipart/form-data">
    <input type="file" name="mypic" accept=".jpg,.jpeg,.png"/>
    <input type="submit" value="Upload"/>
  </form>
</html>
```

Example 2

Write an express js script that accepts **multiple files max number 5** to be uploaded using the multer middleware and saves the files to the specific directory called **"multiple"**.

multiple.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
//CREATE EXPRESS APP
const app = express();

app.use(express.static(__dirname,{index: 'multiple.html'}));

// SET STORAGE
var store = multer.diskStorage({
  destination:"multiple",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-"+Date.now()+".docx")
  }
})
var upload = multer({ storage: store })
app.post('/uploadfile', upload.array('mypic',5), (req,res) => {
  const files = req.files
  if (files) {
    res.set("content-type","text/html")
    for(i of files){
      res.write("<h2>File <span style='color:red'>" + i.originalname + "</span> has been
uploaded </h2>")
    }
    res.send()
  }
})
app.listen(6788);
```

multiple.html

```
<html>
  <form action="/uploadfile" method="post" enctype="multipart/form-data">
    <input type="file" name="mypic" accept=".doc,.docx" multiple>
    <input type="submit" value="Upload"/>
```

```
</form>
</html>
```

In this example, we use **Date.now()** to generate a unique timestamp for each uploaded file. It returns the number of milliseconds since January 1, 1970

limits

- ✓ Set File Size Limits
- ✓ When setting file size limits, we need to specify the maximum file size that can be uploaded. The limits option of Multer can be used to set the maximum file size in bytes.

In the example, we set the maximum file size to 1MB (1,000,000 bytes). You can change this to any value that fits your requirements.

Here's an example of how to set file size limits:

```
const upload = multer({
  storage,
  limits: {
    fileSize: 1000*1000 // 1MB (decimal)
  }
});
```

Or

```
const upload = multer({
  storage,
  limits: {
    fileSize: 1024*1024 // 1MB (binary)
  }
});
```

In this example, we set the maximum file size to 1MB.

The megabyte is one of several multipliers used to represent larger numbers of bytes. For example, a kilobyte (KB) is equal to 1,000 bytes (decimal) or 1,024 bytes (binary). As such, a megabyte is equal to 1,000 KB (decimal) or 1,024 KB (binary).

Example

Write express js script to upload file with size limit of 10 MB to a specific directory named "Data" on the server. And in this folder file must be stored in format of "lju-file.pdf" where "lju" is the field name.

mult1.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
//CREATE EXPRESS APP
const app = express();

app.use(express.static(__dirname,{index:'mult1.html'}))

// SET STORAGE
var storage = multer.diskStorage({
  destination:"Data",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-"+ "File.pdf")
  }
})
var upload = multer({ storage: storage,limits:{fileSize:10*1024*1024} })
app.post('/uploadfile', upload.single('lju'), (req,res) => {
  const file = req.file
  res.send("<h1 style='color:red'>" + file.originalname+" has been Uploaded</h1>")
})
app.listen(6788);
```

mult1.html

```
<html>
  <form action="/uploadfile" method="post" enctype="multipart/form-data">
    <input type="file" name="lju" accept=".pdf"/>
    <input type="submit" value="Upload"/>
  </form>
</html>
```

Example

Write an express js script to configure the **multer** middleware. Perform following tasks.

- 1) Create a **html** file named "file.html". This file contains heading(h3) "Upload your CV" in red color. And, a form with input type file(to browse and select file) and submit(to upload the file).
- 2) Create a js file named "file.js" and link this js and html file to browse html file on "/home" page.
- 3) After uploading a file display message on "/upload" page "(file original name) has been uploaded".
- 4) Save uploaded files to specific directory named "upload". And in this folder file must be stored in format of "lju-file.pdf" where "lju" is the field name.

file.html

```
<body>
  <h3 style="color:red">Upload your CV</h3>
  <form action="/upload" method="post" enctype="multipart/form-data">
    <input type="file" name="lju">
    <input type="submit" value="Submit">
  </form>
</body>
```

file.js

```
// require the installed packages
const express = require('express')
const multer = require('multer');
const app = express();

app.use(express.static(__dirname,{index: "file.html"}));

// SET STORAGE
var storage = multer.diskStorage({
  destination:"upload",
  filename: function (req, file, cb) {
    cb(null, file.fieldname+"-file.pdf")
  }
})

var upload = multer({ storage: storage })
app.post('/upload', upload.single('lju'), (req, res) => {
  const file = req.file
```

`console.log(file)`

```
if (file) {  
    res.send("<h1>File <span style='color:red'>"+ file.originalname + "'</span> has been  
uploaded in <span style='color:red'>" + file.destination + " </span>folder")  
}  
})  
app.listen(6788);
```

In browser

Upload your CV

Chapter 1 .pdf

File 'Chapter 1 .pdf' has been uploaded in upload folder

In console

Output of `console.log(file)`

```
{  
  fieldname: 'lju',  
  originalname: 'Chapter 1 .pdf',  
  encoding: '7bit',  
  mimetype: 'application/pdf',  
  destination: 'upload',  
  filename: 'lju-file.pdf',  
  path: 'upload\\lju-file.pdf',  
  size: 1200577  
}
```

Node mailer

Nodemailer is a module for Node.js applications to allow easy email sending.

Install nodemailer using the following command:

```
npm install nodemailer
```

```
let transporter = nodemailer.createTransport(options[, defaults])
```

Where

- **options** – is an object that defines connection data (see below for details)
- **defaults** – is an object that is going to be merged into every message object. This allows you to specify shared options, for example to set the same *from* address for every message

General options

- **port** – is the port to connect to (defaults to 587 if *secure* is *false* or 465 if *true*)
- **host** – is the hostname or IP address to connect to (defaults to *'localhost'*)
- **auth** – defines authentication data

Authentication

If authentication data is not present, the connection is considered authenticated from the start. Otherwise you would need to provide the authentication options object.

auth is the authentication object

1. **user** is the username
2. **pass** is the password for the user if normal login is used

To start sending emails with Nodemailer, all you need to is:

- Create a transporter object
- Configure the mailoptions object
- Deliver a message with `sendMail()`

The following are the possible fields of an email message:

- ✓ **from** - The email address of the sender. All email addresses can be plain *'sender@server.com'* or formatted *'"Sender Name" sender@server.com'*, see Address object for details
- ✓ **to** - Comma separated list or an array of recipients email addresses that will appear on the To: field
- ✓ **cc** - Comma separated list or an array of recipients email addresses that will appear on the Cc: field

- ✓ **bcc** - Comma separated list or an array of recipients email addresses that will appear on the Bcc: field
- ✓ **subject** - The subject of the email
- ✓ **text** - The plaintext version of the message
- ✓ **html** - The HTML version of the message
- ✓ **attachments** - An array of attachment objects. Attachments can be used for embedding images as well.

Simple example to send mail using nodemailer.

mailer.js

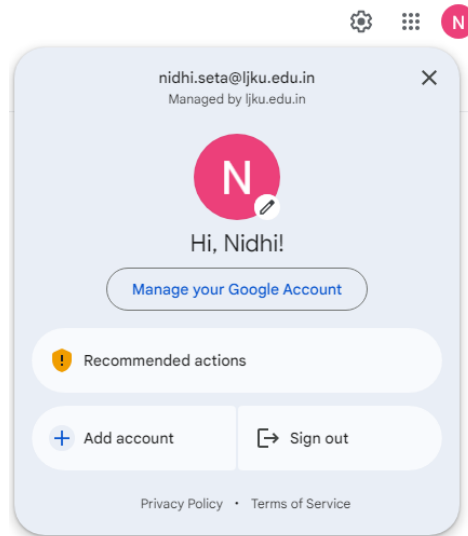
```
var nm = require("nodemailer");

var trans = nm.createTransport({
  host: "smtp.gmail.com",
  port: 465,
  auth: {
    user: "sender@gmail.com",
    pass: "App Password"
  }
});

var mailoption = {
  from: "sender@gmail.com",
  to: "receiver1@gmail.com,receiver2@gmail.com ",
  subject: "Hello",
  text: 'Test mail',
  html: 'Testing node mailer, <h1>Effect of h1</h1>'
};

trans.sendMail(mailoption, (err, info) => {
  if (err) {
    console.error(err);
  }
  console.log(info);
});
```

gmail >>
manage your google account >>



security >>
2 step verification(If set) >>

- Home
- Personal info
- Data & privacy
- Security**
- People & sharing
- Payments & subscriptions
- About

How you sign in to Google

Make sure you can always access your Google Account by keeping this info

2-Step Verification	On since Aug 20, 2022	>
Passkeys and security keys	Start using passkeys	>
Password	Last changed Jun 13	>
Skip password when possible	On	>

app passwords >>

2-Step Verification

Second steps		
Make sure you can access your Google Account by keeping this information up to date and adding more sign-in options		
Passkeys and security keys	Add a security key	>
Google prompt	2 devices	>
Authenticator	Add authenticator app	>
Phone number	085110 80337	>
Backup codes	Get backup codes	>

App passwords		
App Passwords aren't recommended and are unnecessary in most cases. To help keep your account secure, use "Sign in with Google" to connect apps to your Google Account.		
App passwords	2 App passwords	>

Enter App name and create password >>

← App passwords

App passwords help you sign into your Google Account on older apps and services that don't support modern security standards.

App passwords are less secure than using up-to-date apps and services that use modern security standards. Before you create an app password, you should check to see if your app needs this in order to sign in.

[Learn more](#)

Your app passwords

temp	Created on 1:47 AM	🗑️
test	Created on 1:45 AM	🗑️

To create a new app specific password, type a name for it below...

App name

Nodemailer

Create

Example

Perform the following tasks as asked:

- 1) Create a HTML file for response form and this file should be loaded on home('/') page.
•Fields are : Name, Email and Submit button.
- 2) Once Response is submitted, message "Thank you for your response." Will be displayed on page '/response' and also send mail to the entered email id with the submitted response.

app.js

```
const express = require('express');
var nm = require("nodemailer");

const app = express();

// middleware to read form data
app.use(express.urlencoded({ extended: true }));

// serve static files (HTML)
```

```
app.use(express.static(__dirname)); // serves index.html automatically
```

// POST route

```
app.post('/response', (req, res) => {  
  const name = req.body.name;  
  const email = req.body.email;
```

// transporter

```
var trans = nm.createTransport({  
  host: "smtp.gmail.com",  
  port: 465,  
  secure: true,  
  auth: {  
    user: "sender@gmail.com",  
    pass: "your_app_password"  
  }  
});
```

// mail options

```
var mailoption = {  
  from: "sender@gmail.com",  
  to: email,  
  subject: "Response Received",  
  text: `Hello ${name}, your response has been received.`,  
  html: `Hello ${name},<br><br>  
    <h3 style="color:green;">Thank you for your response.</h3>`  
};
```

// send mail

```
trans.sendMail(mailoption, (err, info) => {  
  if (err) {  
    console.error(err);  
    return res.send("Error sending email");  
  }  
  console.log(info);  
  res.send("<h2>Thank you for your response.</h2>");  
});  
});  
  
app.listen(3000, () => {
```

```
console.log("Server running on port 3000");  
});
```

index.html

```
<h2>Response Form</h2>
```

```
<form action="/response" method="POST">
```

```
  Name: <input type="text" name="name" required><br><br>
```

```
  Email: <input type="email" name="email" required><br><br>
```

```
  <input type="submit" value="Submit">
```

```
</form>
```

What is EJS?

What is the "E" for? "Embedded?" Could be. How about "Effective," "Elegant," or just "Easy"? EJS is a simple templating language that lets you generate HTML markup with plain JavaScript. No religiousness about how to organize things. No reinvention of iteration and control-flow. It's just plain JavaScript.

- **Use plain JavaScript**

We love JavaScript. It's a totally friendly language. All templating languages grow to be Turing-complete. Just cut out the middle-man, and use JS!

- **Fast development time**

Don't waste time and attention figuring out arcane new syntax because 'elegance' or how to preprocess your data so it will actually render right.

- **Simple syntax**

JavaScript code in simple, straightforward scriptlet tags. Just write JavaScript that emits the HTML you want, and get the job done!

- **Speedy execution**

We all know how fast V8 and the other JavaScript runtimes have gotten. EJS caches the intermediate JS functions for fast execution.

- **Easy debugging**

It's easy to debug EJS errors: your errors are plain JavaScript exceptions, with template line-numbers included.

- **Active development**

EJS has a large community of active users, and the library is under active development. We're happy to answer your questions or give you help.

EJS lets you write **HTML + JavaScript together** and render dynamic pages using **Express**.

Installation

```
npm install ejs
```

Example

```
const express = require('express');
const app = express();
// set EJS
app.set('view engine', 'ejs');
```

```
app.get('/', (req, res) => {
  res.render('first');
});
app.listen(3000);
```

EJS TAGS

Tag	Description	Example
<code><% %></code>	Scriptlet (logic only, no output)	<code><% if(x>10){ %></code>
<code><%= %></code>	Output (HTML escaped)	<code><%= name %></code>
<code><%- %></code>	Output (unescaped HTML)	<code><%- "Hi" %></code>
<code><%# %></code>	Comment (not executed)	<code><%# comment %></code>
<code><%%</code>	Print literal <code><%</code>	<code><%% Hello %></code>

1. `<% %>` → Scriptlet Tag (Logic Only)

Purpose:

Used for **control flow (if, loop, variables)**

Does NOT print anything

Example:

```
<% let marks = 60; %>
<% if (marks >= 40) { %>
  <p>Pass</p>
<% } else { %>
  <p>Fail</p>
<% } %>
```

Output:

Pass

2. `<%= %>` → Output (HTML Escaped)

Purpose:

Displays data safely (prevents HTML injection)

Example:

```
<% let name = "<b>Rahul</b>"; %>
<p><%= name %></p>
```

Output:

Rahul (shown as text, NOT bold)
Tags are escaped → safe

3. **<%- %> → Output (Unescaped HTML)**

Purpose:

Displays raw HTML (renders tags)

Example:

```
<% let name = "<b>Rahul</b>"; %>
<p><%- name %></p>
```

Output:

Rahul (in bold)

Note:

<%= %> → shows **text only (safe)**

HTML tags are NOT applied

<%- %> → shows **real HTML**

HTML tags ARE applied

4. **<%# %> → Comment Tag**

Purpose:

Add comments that are **not visible in browser**

Example:

```
<%# This is a comment %>
<p>Hello</p>
```

Output:

Hello

5. **<%% %> → Print <% Literally**

Purpose:

Used when you want to show EJS syntax as text

Example:

```
<p><%% This is EJS %></p>
```

Output:

<% This is EJS %>

Example:

Structure

```
project/
|
├── app.js
├── views/
│   └── first.ejs
```

app.js

```
const express = require('express');
const app = express();

app.set('view engine', 'ejs');

app.get('/', (req, res) => {
  res.render('first', { name: "Rahul" });
});

app.listen(3000);
```

views/first.ejs

```
<h1>Hello <%= name %></h1>
```

If you **don't want to use a views folder**, you just need to tell Express where your EJS files are located.

Folder structure

```
project/
|
├── app.js
└── first.ejs  (no views folder needed)
```

You can change the views directory like this:

```
const express = require('express');
const path = require('path');
const app = express();
app.set('view engine', 'ejs');
```

```
// set custom folder (same directory or any folder you want)
app.set('views', path.join(__dirname));
app.get('/', (req, res) => {
  res.render('first', { name: "Rahul" });
});

app.listen(3000);
```

FORM HANDLING WITH EJS + EXPRESS

Example

Develop a web application using Express and EJS that allows a user to enter a student's name and marks through a form. After submission, the application should display the entered details and determine the result: if the marks are 40 or above, show "Pass" in green color; otherwise, show "Fail" in red color.

Create Form (form.ejs)

```
<h2>Student Form</h2>
<form action="/submit" method="post">
  Name: <input type="text" name="name"><br><br>
  Marks: <input type="number" name="marks"><br><br>
  <input type="submit" value="Submit">
</form>
```

Server Code(my.js)

```
const express = require('express');
const app = express();
app.set('view engine', 'ejs');
// to read form data
app.use(express.urlencoded({ extended: true }));
// show form
app.get('/', (req, res) => {
  res.render('form');
});
// handle form
app.post('/submit', (req, res) => {
  const name = req.body.name;
  const marks = req.body.marks;
  res.render('result', { name, marks });
});
```

```
});  
app.listen(3000);
```

Result Page (result.ejs)

```
<h2>Name: <%= name %></h2>  
<h2>Marks: <%= marks %></h2>  
<% if (marks >= 40) { %>  
  <p style="color:green">Pass</p>  
<% } else { %>  
  <p style="color:red">Fail</p>  
<% } %>
```

Example

Write an Express.js application to perform the following tasks:

1) Create an EJS file named contact.ejs that displays a heading "Contact Form" and contains a form with:

- Text input for Name
- Email input for Email
- Submit button

2) Render this EJS file on the /contact route.

3) Handle form submission using the POST method on the /submit route.

4) After form submission, display a message on the browser in the format: "Thank you (Name), we have received your email (Email)." Through ejs file.

Folder Structure

```
project/  
├──  
├── app.js  
├── views/  
│   ├── contact.ejs  
│   └── result.ejs
```

contact.ejs

```
<h2>Contact Form</h2>  
  
<form action="/submit" method="POST">  
  Name: <input type="text" name="name" required><br><br>  
  Email: <input type="email" name="email" required><br><br>  
  <input type="submit" value="Submit">  
</form>
```

app.js (Express Server)

```
const express = require('express');
const app = express();

// set EJS
app.set('view engine', 'ejs');

// middleware to read form data
app.use(express.urlencoded({ extended: true }));

// render contact form
app.get('/contact', (req, res) => {
  res.render('contact');
});

// handle form submission
app.post('/submit', (req, res) => {
  const name = req.body.name;
  const email = req.body.email;
  // render result page with data
  res.render('result', { name, email });
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

result.ejs

```
<h2>
  Thank you <%= name %>, we have received your email <%= email %>.
</h2>
```

Example

Develop a web application using Express and EJS to accept marks of four tests (T1, T2, T3, T4), each out of 25, through a form using the POST method. After submission, display all entered marks in a tabular format along with the total marks, and determine the result: if the total is **35 or more**, display **“Pass” in green color**; otherwise, display **“Fail” in red color**. (result also by ejs file)

```
project/
|
├── app.js
├── views/
│   ├── form.ejs
│   └── result.ejs
```

form.ejs

```
<h2>Enter Marks (Each out of 25)</h2>

<form action="/submit" method="POST">
  T1: <input type="number" name="t1" required><br><br>
  T2: <input type="number" name="t2" required><br><br>
  T3: <input type="number" name="t3" required><br><br>
  T4: <input type="number" name="t4" required><br><br>

  <input type="submit" value="Submit">
</form>
```

result.ejs

```
<h2>Marks Details</h2>
<table border="1" cellpadding="10">
  <tr>
    <th>T1</th>
    <th>T2</th>
    <th>T3</th>
    <th>T4</th>
  </tr>
  <tr>
    <td><%= t1 %></td>
    <td><%= t2 %></td>
    <td><%= t3 %></td>
    <td><%= t4 %></td>
```

```
</tr>
</table>
<br>
<h2>Total: <%= total %></h2>
<% if (total >= 35) { %>
  <h3 style="color:green;">Pass</h3>
<% } else { %>
  <h3 style="color:red;">Fail</h3>
<% } %>
```

app.js

```
const express = require('express');
const app = express();

app.set('view engine', 'ejs');

// middleware for POST data
app.use(express.urlencoded({ extended: true }));

// show form
app.get('/', (req, res) => {
  res.render('form');
});

// handle form
app.post('/submit', (req, res) => {
  const t1 = parseInt(req.body.t1);
  const t2 = parseInt(req.body.t2);
  const t3 = parseInt(req.body.t3);
  const t4 = parseInt(req.body.t4);

  const total = t1 + t2 + t3 + t4;

  res.render('result', { t1, t2, t3, t4, total });
});
app.listen(3000);
```

Example

Create a basic web application using Node.js, Express.js, and EJS to display user information on a webpage.

The application should:

- Configure EJS as the view engine
- Create a route `/users` in the server
- Inside this route, define a list of users (each user should have name, email, and age)
- Pass this user data from the server to an EJS file
- Use EJS to loop through the data and display it in an HTML table

my.js

```
const express = require('express');
const app = express();

app.set('view engine', 'ejs');

// sample JSON data
app.get('/users', (req, res) => {
  const users = [
    { name: "Rahul", email: "rahul@gmail.com", age: 22 },
    { name: "Priya", email: "priya@gmail.com", age: 25 },
    { name: "Amit", email: "amit@gmail.com", age: 30 }
  ];

  res.render('my', { users });
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

my.ejs

```
<h2>User List</h2>

<table border="1" cellpadding="10">
  <tr>
    <th>Name</th>
    <th>Email</th>
    <th>Age</th>
  </tr>
```

```
<% for(let u of users) { %>
  <tr>
    <td><%= u.name %></td>
    <td><%= u.email %></td>
    <td><%= u.age %></td>
  </tr>
<% } %>
</table>
```

User List

Name	Email	Age
Rahul	rahul@gmail.com	22
Priya	priya@gmail.com	25
Amit	amit@gmail.com	30